Systolic Array GEMM Accelerator on the Arty A7 FPGA

Spring 2025 ECE 56800 Project Final Report

The Dot Product

Maasil Wani (Lead), Linda Mendez, Wayne Hsieh, Tyler Lee, Niels van Ritbergen, Daniel Arnold

ABSTRACT

Efficient matrix multiplication is the computational backbone of modern computing. However, traditional general purpose processors often encounter a "memory wall", where performance is limited by the latency of memory accesses instead of arithmetic throughput. This proposal outlines the design of a 4x4 INT8 Systolic array on a Arty A7-100T FPGA. This specialized architecture where the data flows through a network of processing elements, ensures that each operand is only loaded from memory only once, reducing bus contention.

1 INTRODUCTION

As neural networks, scientific computing, and digital signal processing continue to advance, the underlying operation which powers these workloads, matrix multiplication, is reaching a performance limit. Matrix operations are highly dependent on data interfacing between memory and the arithmetic units to be processed. A typical matrix multiply on a general-purpose processor involves elements of each matrix being read from memory multiple times. Instead of being able to constantly perform arithmetic, the processor needs to spend additional time waiting for data to arrive from memory [5]. In other words, the overall computational throughput becomes reliant on memory speed rather than the computational speed of the arithmetic units themselves.

2 BACKGROUND

Rather than modify the hardware, an improvement over the basic matrix multiply is to optimize the software to take better advantage of the system's cache size–reducing cache misses and decreasing the number of memory accesses. This can be done by separating the matrices into small tiles so that values can be reused more often without memory reads [6]. This approach requires more effort by the programmer to perform the optimization. The measured improvement of this approach is also directly correlated to the cache size–a smaller cache size will inherently require more memory operations and have a lower performance improvement.

Another potential solution is to widen the datapath as seen in SIMD (single instruction, multiple data) processing. Instead of having an instruction operate on only a single set of values, widening the datapath to allow for computation on several sets of values (vector) at once increases throughput [3]. However, this approach is still reliant on cache sizes as each vector still needs to be read. For larger matrices, not all data will fit in the cache and instead need to be read from memory.

3 PROBLEM STATEMENT

General purpose processors and SIMD- based vector processors are inefficient for high throughput matrix operations due to the Von Neumann bottleneck. This bottleneck refers to the use of a single bus to process both instruction and data operations, meaning that data has to constantly be transferred and moved between the cache and the ALU [4]. The result is high power consumption and many idle cycles while waiting to fetch from memory. This project addresses the problem of the Von Neumann bottleneck by creating an embedded hardware accelerator that minimizes memory bandwidth through data reuse.

4 PROPOSED APPROACH

A systolic array for matrix multiplication addresses the aforementioned performance limitations from memory accesses by storing all operands and partial results within the structure. Unlike a traditional matrix multiply, each element of the matrices being multiplied need to be loaded only once. Once the array is full, every processing element is computing useful data for the final result.

To implement the systolic array onto an FPGA, a design that passes not only in simulation, but also physically, is necessary. This design can be divided into three sections:

Systolic Array: This is the computational heart of the design. The focus of this section is strictly on the datapath, ensuring that the processing elements and data pipeline operate seamlessly. Responsibilities include:

Processing Element(PE): Implementing the localized multiply-accumulate units in SystemVerilog. Each PE must manage its own internal registers to hold partial sums and cleanly pass data to adjacent PEs in the grid. Data Skewing and Aligment Logic: Designing the delay elements (shift registers) required to stagger the matrix operands. Because data flows through the array like a wavefront, the row and column inputs must be systematically delayed so that the correct elements intersect at the correct PE at the exact right clock cycle. Input and Output FIFOs: Instantiating and managing the FIFOs that buffer data entering the array and capture the computed partial/final products exiting the array. This acts as a crucial elastic buffer between the rigid timing of the systolic grid and the controller.

Processor-Systolic Array Controller: This section bridges the gap between static memory and the active computation grid. This team handles the control path, memory addressing, and data orchestration. Responsibilities include:

Developing the core state machine to sequence the entire matrix multiplication operation. Typical states include Idle, Load Matrix A, Load Matrix B, Compute/Stall, and Drain Results. BRAM Interface and Address Generation: Creating the logic to read from and write to Block RAM (BRAM). This involves calculating the correct read addresses based on matrix dimensions, handling the read latency of the BRAMs, and routing that data into the input FIFOs of the systolic array. Integration Testing: Combining the Controller with the Systolic Array to verify that data can be correctly fetched

from simulated BRAM, processed, and written back to memory seamlessly.

MicroBlaze and Vivado Integration: The custom RTL was integrated into a full SoC on the Arty A7-100T using Vivado's block design environment. A Clocking Wizard generates a 100 MHz system clock, which drives a MicroBlaze V softcore processor [2] connected to its local instruction and data memories. An AXI Smart-Connect routes the processor's data bus to three AXI slaves: an AXI BRAM Controller, an AXI UartLite peripheral, and the custom Control Registers AXI-Lite IP [1]. The BRAM Controller provides the processor with AXI access to a dual-port Block RAM , whose second port is connected directly to the accelerator wrapper [7], allowing the accelerator to read input matrices and write results without processor involvement. A MicroBlaze Debug Module (MDM) was included for JTAG debugging during development.

The Control Registers acts as the hardware-software interface, exposing a memory-mapped register set over AXI-Lite. The processor writes the source and destination BRAM addresses and asserts a go signal to launch a computation. The accelerator acknowledges with an ack pulse, drives busy while running, and raises a sticky done flag when complete, which the processor polls over UART.

Hardware validation was performed entirely in software running on the MicroBlaze. A reference software matrix multiply was executed alongside the hardware accelerator on the same inputs, with results compared and reported over UART. Cycle-accurate timing was measured using the RISC-V `rdcycle` CSR, capturing both software and hardware execution times for a 4×4 multiply and a 16×16 tiled GEMM using repeated 4×4 accelerator calls with software accumulation of partial results.

4.1 C Golden Model

Before developing and implementing firmware on the FPGA, a model was developed to simulate the the desired behavior. This model can be taken as a proof of concept, quickly and easily developed using the C language, and is useful for breaking down the project into manageable parts before any firmware is developed. Future development on the FPGA should match the behavior of this C-model.

To validate the results produced by the systolic array, we implemented both a C-based systolic array model and a reference (classical) matrix multiplication. We used the classical multiplication to verify the final output, and then used the C systolic array model to generate the intermediate (partial) results. This allowed us to confirm that the partial results produced by our Verilog implementation were correct.

Beyond the mathematical datapath, the C model simulates the Processor-Systolic Array Controller and its interaction with the surrounding system. The simulation is driven by a main clock loop that evaluates system state on a cycle-by-cycle basis, mirroring the synchronous nature of the FPGA. A dedicated tick_fsm() function evaluates the state of the control registers to transition between operational states (e.g., IDLE, LOAD, DONE_STATE). This provides a verified state transition table that the SystemVerilog FSM must exactly replicate. To simulate the reading and writing of Block RAM, the model utilizes a file-flow architecture. State transitions trigger file I/O operations—such as reading input matrices from a simulated bram.txt into the array during the LOAD state, or draining the final mock_results out to a bram_out.txt upon reaching the DONE_STATE. This validates the memory addressing logic before integrating actual BRAM IP.

4.2 SystemVerilog RTL Design

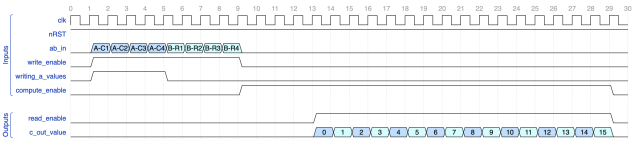

Figure 1: Systolic Array Timing Diagram

The systolic array begins loading data when the controller raises a write_enable signal. This signal tells the top-level array module to begin loading the FIFOs with incoming data from the CPU. Which FIFO is being loaded is determined by the writing_a_values signal. If this signal is high, the FIFOs corresponding to the A matrix are loaded, otherwise the FIFOs corresponding to the B matrix are loaded.

Entries to the A matrix are added to 4 FIFOs at a time, with each set of values corresponding to a matrix column. Similarly, the B matrix data is stored in the systolic array module using 4 FIFOs, but unlike the A matrix, the data loaded in at each cycle corresponds to a row. This effectively means that the matrices data stored in memory is pre-transposed, so extra cycles do not need to be spent on this transformation.

Immediately after these 8 load cycles are complete, the computation phase of the systolic array can begin. The data compute phase begins with the assertion of a compute_enable flag. This tells the array that all data has been loaded and it is safe to compute based on the current data in the FIFOs. After 4 cycles of no output, the systolic array asserts a read_enable flag, indicating to the controller that the c_out_value bus contains valid entries that should be returned to the CPU.

The read_enable flag remains asserted for 16 following cycles, corresponding to one cycle for each entry in the final matrix (4x4). All of the computed data is returned by the 32-bit c_out_value bus, one entry at a time. This bus is 32-bits since the product of two 8-bit values can create as high as a 32-bit value.

In an effort to reduce the number of cycles needed, there is a significant overlap between data computation in the systolic array, and the data transfer to the CPU. In other words, while the systolic array is still computing a matrix multiply, any completed entries are sent to the CPU. This increases the performance differential between a CPU performing a matrix multiply and a systolic array performing the same multiply.

To interface the RTL design with the processor, values are read from the BRAM block memory using Port B access and passed to the systolic array through the PE controller in the above process. When the systollic array signals that it is done, the compute values will be passed back through the PE controller to the BRAM block memory as the values are outputted.

In the above PE controller, there are two components: a FSM to control the systolic array and a BRAM to FSM interface to read and write the registers in the block memory and pass the values to and from the FSM and the systolic array. This FSM accepts the systolic array output signals and responds with the relevant behavior described above. The BRAM interface accepts register address data and trigger signals from the MicroBlaze processor and provides the final matrix register address and done signals.

The FSM.sv module serves as the primary hardware controller for the 4x4 systolic array, systematically sequencing the execution of a matrix multiplication operation across five distinct states. Upon receiving an active go_sig in the IDLE state, the controller safely latches the full 4x32-bit input matrices (A and B) into internal registers to prevent data corruption during processing. It then initiates an eight-cycle sequential data-loading phase: spending four cycles in LOAD_A to stream Matrix A into the array by asserting the writing_a_values and write_enable signals, immediately followed by four cycles in LOAD_B to stream Matrix B. Once the operands are loaded, the state machine transitions to COMPUTE_NO_OUTPUT for a four-cycle priming period, activating compute_enable to allow the initial wave of data to propagate through the processing elements without capturing garbage outputs. Finally, the controller enters the COMPUTE_OUTPUT state for sixteen cycles, maintaining the computation pipeline while actively monitoring the s_read_enable flag; when valid data reaches the array's boundary, it captures the c_out_value, asserts m_read_enable, and routes the completed data to matrix_S_output before seamlessly returning to the IDLE state to await the next operation.

The `ctrl_regs_fsm.sv` is a custom AXI-Lite slave exposing a memory-mapped register set to the MicroBlaze. It implements independent read and write FSMs, providing registers for the source addresses of matrices A and B, the destination address, and a go bit (0x00–0x10). Three read-only status registers expose the accelerator's busy and done signals to the processor (0x18–0x1C). The go register self-clears on accelerator acknowledgement, and done is held high until the next go assertion, allowing safe software polling.

4.3 FPGA Physical Implementation

To test the physical implementation of the systolic array itself on the FPGA, a wrapper was used to store the pre-transposed matrices to feed directly into the module. Internal signals for waveform viewing were then captured using an Internal Logic Analyzer (ILA), which was added to the netlist during synthesis.

Figure 3 shows the full Vivado block design of the system. The MicroBlaze V processor connects via AXI SmartConnect to three AXI slaves: the AXI BRAM Controller, AXI UartLite, and the custom Control Registers. The dual-port Block RAM is shared between the processor (Port A via AXI) and the accelerator wrapper (Port B directly), enabling matrix transfers without processor involvement. Table 1 shows the corresponding memory map as configured in Vivado. The shared BRAM is mapped at `0xC000_0000`, the Control Registers control registers at `0x8000_0000`, and the UartLite peripheral at `0x4060_0000`. MicroBlaze local instruction and data memories occupy the base address at `0x0`.

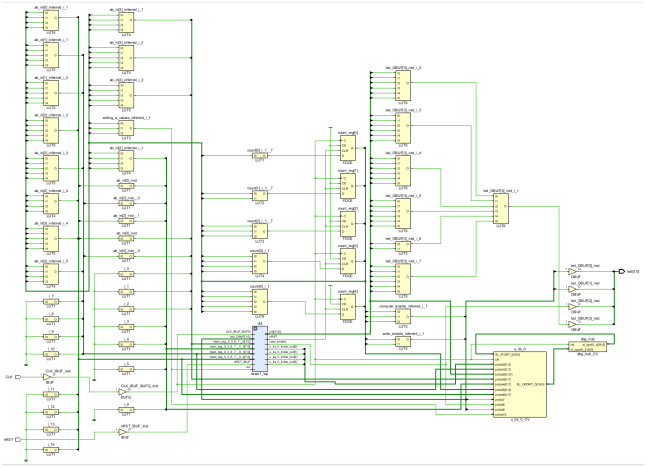

Figure 2: Systolic Array FPGA Test Wrapper Schematic

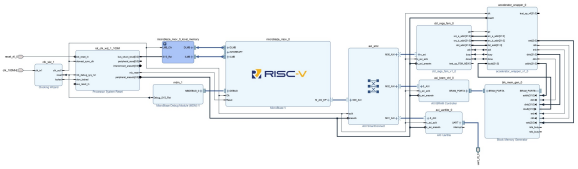

Figure 3: MicroBlaze V memory map as configured in Vivado.

Peripheral	Base Address	Size
Local Memory (DLMB/ILMB)	0x0000_0000	16K
Control Registers	0x8000_0000	4K
AXI BRAM Controller	0xC000_0000	16K
AXI UartLite	0x4060_0000	64K

Table 1: MicroBlaze V memory map as configured in Vivado.

4.4 Vitis Software Development

The FPGA bitstream was exported from Vivado to Vitis, where all software was developed and deployed to the MicroBlaze V processor. A bare-metal application was created using the generated hardware platform, which provided all peripheral base addresses through `xparameters.h`.

Four tests were developed and run on the hardware:

Accelerator Correctness Test: Known 4×4 INT8 matrices were written to shared BRAM, the accelerator was triggered, and the results were read back and printed over UART to verify correct end-to-end operation.

Software Matrix Multiply: A standard matrix multiply was implemented in C and timed using the RISC-V `rdcycle` CSR, providing a baseline for comparison against the hardware accelerator.

Hardware 4×4 Benchmark: The accelerator was timed on a single 4×4 matrix multiply using `rdcycle`, with results printed over UART alongside the software baseline.

16×16 Tiled GEMM: A 16×16 matrix multiply was implemented by decomposing the problem into 64 4×4 sub-block accelerator calls.

For each call, the relevant sub-blocks are packed into shared BRAM, the accelerator is triggered, and the partial results are accumulated in software. The total execution was timed and compared against a 16×16 software reference.

5 EVALUATION

Beginning with separated testing for each component of the design, the systolic array module was successfully able to function in isolation. For the purposes of testing, and to number the entries of each matrix for viewing, the following matrix multiply was tested.

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} = \begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix}$$

Figure 4: Matrix Multiplication for Systolic Array Test

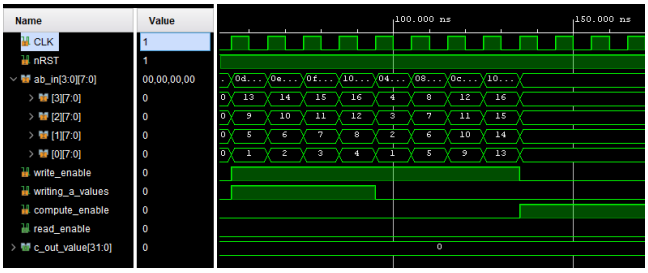

Figure 5: Systolic Array Load Procedure Waveform

Similar to Figure 1, Figure 5 shows the systolic array correctly computing a pre-transposed pair of matrices. This was done in simulation using Vivado with the enable control set by a testbench. Matrix A is sent in column by column, while Matrix B is sent in row by row.

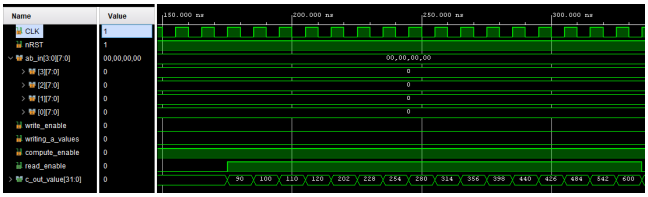

Figure 6: Systolic Array Return Procedure Waveform

Following the load procedure, the data return shown in Figure 6 is also correct. Following 4 idle cycles with no output, the following 16 cycles display each index in the resultant matrix, starting with row 1, column 1, and ending with row 4, column 4.

The same timing and results can also be seen on the FPGA implementation.

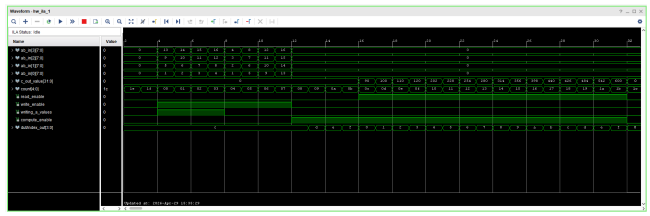

Figure 7: Systolic Array FPGA ILA Waveform

Module	LUTs	Registers	F7 Muxes
Systolic Array	2157	766	48
FSM Controller	117	264	0
Accelerator Wrapper	2447	1872	48
Control Registers	48	136	0
MicroBlaze V	1606	1317	88
MDM	143	199	5
AXI SmartConnect	397	463	1
AXI BRAM Controller	150	169	0
AXI UartLite	92	111	10
Total	**4917**	**4329**	**157**

Table 2: FPGA Resource Utilization for Complete Design on Arty A7-100T.

Table 3 summarizes the measured performance of the hardware accelerator against the software baseline for both matrix sizes. All results were measured on the fully integrated system, comprising the MicroBlaze V processor, AXI interconnect, shared BRAM, control registers, and the systolic array accelerator running on the Arty A7-100T at 100 MHz.

Test	Cycles	Speedup
SW 4×4 Matrix Multiply	7,720	1×
HW 4×4 Matrix Multiply	164	47×
SW 16×16 Matrix Multiply	496,312	1×
HW 16×16 Tiled GEMM	235,021	2.1×

Table 3: Performance comparison between software and hardware matrix multiply at 100 MHz on the Arty A7-100T. Speedup is relative to the software baseline at each matrix size.

6 DISCUSSION

The 4×4 hardware accelerator achieves a 47× speedup over the software baseline. This is expected, as the systolic array computes all 16 output elements in parallel through its pipeline, completing the full multiply in 164 cycles including AXI transaction overhead. The software implementation executes 64 multiply-accumulate operations sequentially with loop and memory access overhead, making the hardware advantage significant at this scale.

The 16×16 tiled result tells a different story. Despite the same underlying hardware, the speedup drops to 2.1×. The reason is that

the 16×16 blocked implementation requires 64 separate accelerator calls, each incurring the full AXI overhead of writing input sub-blocks to BRAM, issuing control register writes, polling the done flag, and reading back 16 result words. Estimating the pure compute time as 64 calls at 164 cycles each gives approximately 10,496 cycles, meaning roughly 95% of the total 235,021 hardware cycles is AXI communication overhead rather than computation. Meanwhile, the software 16×16 multiply scales cubically in the number of operations, growing from 64 to 4,096 multiply-accumulates, which is why the hardware still wins despite the overhead.

This highlights the central limitation of the current design: the processor is the bottleneck, not the systolic array. Each sub-block call requires the processor to manually pack data, issue AXI writes, poll for completion, and read results. For larger matrices this overhead grows linearly with the number of tiles while the compute grows cubically, meaning the hardware advantage would recover at larger matrix sizes. A DMA engine capable of autonomously feeding the accelerator with successive sub-blocks without processor involvement would significantly reduce this overhead. Additionally, moving from a polling-based done signal to an interrupt-driven model would eliminate idle cycles spent waiting for completion.

The design is also currently limited to 4×4 INT8 inputs with INT32 accumulation. While this is sufficient for demonstrating the concept, a larger array or support for wider data types would be necessary for practical workloads such as neural network inference.

7 POTENTIAL IMPACTS

Accelerators have the potential to operate in low bandwidth systems, reduce power consumption, and decrease latency. This directly translates to systems requiring fewer resources and providing faster and more efficient operation. Current technologies and concepts such as SoCs, AI, machine learning, and autonomous systems can be improved with faster response to inputs and disturbances, smaller systems, and reduced power consumption, especially for energy intensive applications.

In the non-technical realm, the benefits may be potentially observed in physical aspects such as smaller consumer electronics, extended battery life, and self driving cars and robots. Digital tools, too, may potentially be improved, such as photo and video editing, animation, AI interactions, and real time translation or item detection.

8 CHANGE OF PLAN

There were no significant deviations from the original proposal. As a team, work was divided according to team members' skills and project subcomponent complexity. The greatest extent of deviation occured in the timeline, with full project integration falling behind schedule due to issues with completely interfacing the PE controller with the other two components. No other deviations occured.

9 EMBEDDED SYSTEMS ASPECTS

This project is an embedded hardware-software co-design built around the Arty A7 FPGA. A MicroBlaze V softcore processor executes bare-metal C software, manages memory-mapped control registers, writes input matrices to shared BRAM, launches the accelerator, polls completion status, and verifies the output over UART. The compute-intensive matrix multiplication kernel is offloaded to custom SystemVerilog hardware.

The design reflects several embedded systems concepts, including memory-mapped I/O, AXI-Lite peripheral control, shared memory communication, hardware/software partitioning, polling-based synchronization, and resource-constrained FPGA implementation. The use of dual-port BRAM allows the processor and accelerator to share data while avoiding direct bus contention during accelerator execution. The 16×16 tiled GEMM further demonstrates a common embedded accelerator tradeoff, although the systolic array provides high compute throughput, software-managed data movement and repeated accelerator launches can dominate total runtime for larger workloads.

10 TASK BREAKDOWN

Niels van Ritbergen - Prototyped a systolic array module in C to develop a cycle-by-cycle model for future accuracy checking. Wrote the FIFO, PE, and systolic array RTL modules. Verified and modified the design using preliminary results from Vivado simulations, both pre and post-synthesis. Integrated an ILA into the netlist and tested the isolated systolic array design on an FPGA.

Linda Mendez – Implemented the systolic array in C as a golden reference model, along with classical matrix multiplication to verify final results. Developed the top module to connect the skew FIFOs and integrate them with the systolic array. Wrote testbenches to simulate the top-level systolic array module and ran simulations with various input matrices to validate the design by comparing outputs against the C golden model.

Tyler Lee - Generate testbenches for other team members to simulate modules for PE controller and BRAM interfaces based on Team 1 and Team 3 expected behavior. Researched interfaces for accessing BRAM for both Microblaze and FPGA firmwrae modules and informed the team to use AXI for Microblaze and Port B interface for the FPGA modules.

Maasil Wani - Designed the overall system architecture and SoC integration in Vivado, connecting the MicroBlaze V processor, AXI SmartConnect, AXI BRAM Controller, dual-port Block RAM, and custom accelerator IP. Designed the Control Registers AXI-Lite slave, exposing the accelerator control interface to the processor, and developed an early controller stub defining the hardware interface contract for the team. Integrated AXI UartLite for serial output and wrote MicroBlaze software, including a reference software matrix multiply, benchmarking via the RISC-V `rdcycle`, and a 16×16 tiled GEMM implementation using repeated 4×4 accelerator calls with software partial-result accumulation.

Daniel Arnold - Created the framework for integration between teams using a top module to connect the pe controller and systolic array and a wrapper file between the pe controller and Microblaze. Developed a BRAM loader for the pe controller and assisted in BRAM integration and testing. Made log output and file flow files used in intial pe controller testing to verify functionality.

Wayne Hsieh - created the C model to simulate the state transition of the PE controller, and tested the top level of the C model to simulate reading and writing data to the BRAM. Developed the FSM in SystemVerilog based on the waveforms from the systolic array.

Made sure that the FSM will sync up with the input and outputs of the systolic array.

11 RELATION TO OTHER PROJECT

(Linda Mendez) – As part of my learning process, my advisor asked me to implement a systolic array, which is related to this project. Implementing it will help me better understand how the systolic array works.

REFERENCES

[1] AMD. 2024. AXI4-Lite Interface. https://docs.amd.com/r/en-US/pg202-mipi-dphy/AXI4-Lite-Interface. MIPI D-PHY LogiCORE IP Product Guide (PG202), Version 4.3. Accessed: 2026-05-08.

[2] AMD. 2025. MicroBlaze Processor Reference Guide (UG984). https://docs.amd.com/r/en-US/ug984-vivado-microblaze-ref/Introduction. Version 2025.2. Accessed: 2026-05-08.

[3] ARM Ltd. 2020. Introducing NEON - SIMD. https://developer.arm.com/documentation/den0013/0400/Introducing-NEON/SIMD.

[4] John Backus. 1978. Can programming be liberated from the von Neumann style? a functional style and its algebra of programs. *Commun. ACM* 21, 8 (Aug. 1978), 613–641. doi:10.1145/359576.359579

[5] H. T. Kung. 1982. Why Systolic Architectures? https://www.eecs.harvard.edu/~htk/publication/1982-kung-why-systolic-architecture.pdf.

[6] E. L. Talmage. 2024. Divide & Conquer: Square Matrix Multiplication. https://eg.bucknell.edu/~elt006/311/F24/6-DC-SMM.pdf.

[7] Ween's Lab. 2024. Part 7: FPGA Memory. https://weenslab.gitbook.io/pages/fpga-tutorials/zynq-fpga-tutorial/part-7-fpga-memory. Accessed: 2026-05-08.